

Resumen Detallado:

Almacenamiento y Procesamiento en Hadoop (HDFS, YARN, MapReduce)

Contexto del Caso Práctico

Roberto, responsable de tecnología en una empresa de transporte, necesita modernizar su infraestructura para aprovechar los datos de vehículos (mantenimiento predictivo, optimización de rutas, monitorización en tiempo real). Tras evaluar tecnologías, decide implementar **Hadoop** debido a su capacidad para manejar grandes volúmenes de datos (estimados en **40 TB netos** para los próximos 3 años). Este resumen se centra en el **Core de Hadoop: HDFS** (almacenamiento), **YARN** (gestión de recursos) y **MapReduce** (procesamiento).

1. HDFS (Hadoop Distributed File System)

1.1. Introducción y Características

- **Sistema de archivos distribuido:** Ofrece una interfaz unificada aunque funcione en múltiples nodos.
- **Hardware commodity:** Diseñado para ejecutarse en hardware estándar, tolerante a fallos.
- **Optimizado para grandes volúmenes:** Ideal para **ficheros grandes** (ej., bloques de **128 MB por defecto**) y operaciones masivas de lectura/escritura.
- **Escalabilidad horizontal:** Soporta petabytes y miles de nodos, con acceso concurrente.

- **Modelo "write-once, read-many"**: Los archivos no se modifican tras escribirse, pero pueden leerse múltiples veces.
- **Sin restricciones de datos**: Almacena datos estructurados, semiestructurados o no estructurados (imágenes, vídeos, logs).

1.2. Estructura y Funcionamiento

- **Jerarquía de archivos**: Similar a sistemas Unix/Linux, con directorios, subdirectorios y archivos (ej., `/tmp/hive/archivo.csv`).
- **Metadatos**: Para cada archivo almacena:
 - Fecha de modificación.
 - Tamaño.
 - Propietario y grupo.
 - Permisos (sistema Unix: `rw-r--r--`).
- **Tolerancia a fallos mediante replicación**:
 - Los archivos se dividen en **bloques** (128 MB por defecto).
 - Cada bloque se replica en **múltiples nodos** (factor de replicación por defecto: **3**).
 - Si un nodo falla, los datos permanecen accesibles desde otras réplicas.
- **Configuración flexible**:
 - El tamaño de bloque y factor de replicación pueden ajustarse (a nivel de archivo o directorio).
 - Un factor alto mejora la disponibilidad pero penaliza escrituras y consume más espacio.

1.3. Arquitectura de HDFS

- **Namenode (Maestro)**:
 - Gestiona **metadatos** (estructura de archivos, ubicación de bloques, replicación).
 - Coordina operaciones de lectura/escritura.
- **Punto crítico**: Requiere hardware robusto (RAID, fuentes de alimentación redundantes).
- **Secondary Namenode**:

- **No es un respaldo activo**, sino un asistente que reduce el tiempo de recuperación.
 - Realiza **puntos de control (checkpoints)** de los metadatos, combinando `FsImage` (instantánea del sistema) y `EditLog` (registro de cambios).
 - **Datanodes (Nodos de datos):**
 - Almacenan los **bloques de datos**.
 - Reportan su estado al Namenode periódicamente.
-

2. YARN (Yet Another Resource Negotiator)

(Aunque no se detalla en el texto, se menciona como parte del Core de Hadoop) - **Gestor de recursos y procesos:** Coordina la ejecución de aplicaciones (como MapReduce) en el clúster. - **Separa la gestión de recursos del modelo de procesamiento**, permitiendo ejecutar múltiples cargas de trabajo (ej., Spark, Hive) simultáneamente.

3. MapReduce

(Mencionado como modelo de programación para procesamiento) - **Paradigma de programación paralela:** Divide tareas en: - **Map:** Filtra y transforma datos en pares clave-valor. - **Reduce:** Agrega resultados por clave. - **Ejemplo práctico:** Procesar logs de sensores de vehículos para calcular promedios de consumo por ruta.

Aplicación al Caso de Roberto

- **Volumen de datos:** 8 TB iniciales + 10 TB/año → **HDFS** es ideal por su escalabilidad y tolerancia a fallos.
- **Replicación:** Factor 3 asegura disponibilidad ante fallos en vehículos o servidores.
- **Procesamiento:** **MapReduce** (junto con YARN) permitirá analizar datos de sensores y GPS para:

- Predecir averías (mantenimiento predictivo).
 - Optimizar rutas en tiempo real.
 - **Conclusión:** Hadoop Core proporciona la base para construir una plataforma **Big Data** robusta, escalable y económica.
-

Palabras clave: HDFS, bloques, replicación, Namenode, Datanode, MapReduce, YARN, tolerancia a fallos, escalabilidad.

Resumen Detallado: HDFS - Arquitectura, Funcionamiento y Uso

1. Recuperación y Mantenimiento del Estado del Sistema de Archivos

FsImage y EditLog

- **FsImage:** Es una instantánea del sistema de archivos (metadatos: nombres de archivos, estructura de directorios, ubicación de bloques).
- **EditLog:** Registro de todas las operaciones de modificación (crear, borrar, renombrar) que ocurren después de tomar la FsImage.
- **Proceso de Arranque (NameNode):** Al iniciarse, el NameNode fusiona la FsImage con el EditLog para reconstruir el estado completo y actualizado del sistema de archivos en memoria.

Secondary NameNode

- **Función Principal:** No es un nodo de respaldo activo. Su cometido es **aliviar la carga del NameNode** periódicamente fusionando la FsImage y el EditLog en

una nueva FsImage consolidada. Esto evita que el EditLog crezca demasiado y reduce significativamente el tiempo de arranque del NameNode tras un reinicio o fallo.

- **Características:**

- Se ejecuta en una máquina **diferente** al NameNode principal para sobrevivir a sus caídas.
- Suele tener requisitos de hardware similares a los del NameNode.

2. DataNode: Almacenamiento y Gestión de Bloques

Funciones Principales

1. **Almacenar y servir los bloques de datos** (típicamente de 128 MB) que componen los archivos en HDFS.
2. **Desconocimiento del contexto:** Solo conoce los bloques que almacena, no a qué archivo pertenecen ni dónde están los demás bloques de ese archivo. Esta información global solo la tiene el NameNode.

Mecanismos de Comunicación e Integridad

- **Reporte de Bloques:** Periódicamente (inicialmente cada hora), cada DataNode envía al NameNode un listado de los bloques que almacena, permitiendo al NameNode mantener un mapa de ubicaciones actualizado.
- **Checksum (Suma de Verificación):**
 - Para cada bloque, el DataNode calcula y almacena un valor checksum (función matemática del contenido).
 - Al leer el bloque, se recalcula el checksum y se compara con el almacenado. Si no coinciden, se detecta que el bloque está corrupto.
- **Latido (Heartbeat):**
 - Mensaje corto que el DataNode envía al Nameode cada **3 segundos** (por defecto) para indicar que está operativo.
 - Si un DataNode no envía latidos en **10 minutos**, el NameNode lo considera fuera de servicio.

- **Consecuencia:** El NameNode programa la creación de nuevas réplicas de los bloques alojados en ese DataNode fallido en otros DataNodes activos, para mantener el factor de replicación definido.

Hardware

- Se ejecuta en los nodos worker.
- Característica principal: **Gran cantidad de discos** (típicamente, número de núcleos de CPU totales menos uno o dos).
- Menor sofisticación en resiliencia comparado con el NameNode.

3. Funcionamiento: Lectura y Escritura

Principio Fundamental: Inmutabilidad

- Los datos en HDFS son **inmutables**. No se pueden modificar una vez escritos.
- Solo se permite **añadir** contenido (append).
- Para "modificar" un carácter en un archivo, HDFS debe reescribir el bloque completo que lo contiene. Esto, unido al gran tamaño de bloque, hace que HDFS tenga un rendimiento pobre para operaciones aleatorias o pequeñas. Está optimizado para **archivos grandes y lecturas/escrituras secuenciales masivas**.

Proceso de Lectura

1. El cliente pregunta al **NameNode** por la lista de bloques del archivo y los DataNodes que los contienen.
2. El NameNode devuelve la información, ordenando los DataNodes por proximidad al cliente para optimizar el acceso.
3. El cliente se **comunica directamente con cada DataNode** (en el orden recibido) para leer los bloques secuencialmente.
4. **El NameNode no interviene en la transferencia de datos**, evitando convertirse en un cuello de botella.

Proceso de Escritura

1. El cliente solicita al **NameNode** la creación de un nuevo archivo.

2. El NameNode verifica permisos y si el archivo no existe. Si todo es correcto, responde afirmativamente.
3. El cliente divide el archivo en bloques.
4. Por cada bloque, el cliente pregunta al NameNode por una lista de DataNodes donde escribirlo.
5. El cliente escribe el bloque **directamente en el primer DataNode** de la lista.
6. Ese primer DataNode se encarga de replicar el bloque al segundo DataNode de la lista, y este al tercero, formando una **cadena de replicación en pipeline**.
7. Una vez confirmada la escritura en todos los DataNodes de la cadena, se notifica al cliente para que proceda con el siguiente bloque.
8. **Nuevamente, el NameNode no recibe los datos**, solo gestiona la ubicación de los bloques.

4. Implicaciones del Factor de Replicación

El factor de replicación (número de copias de cada bloque) es crucial y tiene múltiples implicaciones:

Factor de Replicación ALTO (ej., 3)

- **Ventajas:**
 - **Mejora la tolerancia a fallos** (pueden caerse más DataNodes sin perder datos).
 - **Mejora la velocidad de lectura** (el cliente puede leer de múltiples fuentes y elegir la más cercana/desocupada).
- **Desventajas:**
 - **Reduce la velocidad de escritura** (hay que escribir cada bloque en más nodos).
 - **Reduce la capacidad útil de almacenamiento** del clúster (los datos ocupan el triple).

Factor de Replicación BAJO (ej., 1)

- **Ventajas:**
 - **Mayor velocidad de escritura.**
 - **Mayor capacidad útil de almacenamiento.**

- **Desventajas:**
 - **Mayor riesgo de pérdida de datos.**
 - **Posible menor velocidad de lectura** (menos fuentes disponibles).

Cálculo de Capacidad y Reglas Prácticas

- **Capacidad Bruta:** Suma de la capacidad de todos los discos en los DataNodes.
- **Capacidad Neta:** Se reduce por el factor de replicación y por un espacio reservado (30-40%) para datos temporales de aplicaciones y logs.
 - **Ejemplo:** Clúster de 20 nodos, 12 discos de 3TB cada uno.
 - Capacidad bruta: 720 TB.
 - Con replicación 3: 240 TB para datos de usuario.
 - Reservando 30%: Capacidad neta útil $\approx$ **168 TB**.
- **Reglas para elegir el factor:**
 - **Datos temporales/no críticos:** Replicación baja (1 o 2).
 - **Datos críticos/muy accedidos:** Replicación alta (3 o más, incluso igual al número de DataNodes si el archivo es pequeño).
 - **Datos generales:** Usar el valor por defecto (normalmente 3).

5. Uso e Interfaces de HDFS

Operaciones Soportadas

Similares a sistemas Unix: gestión de archivos (leer, escribir, borrar), gestión de directorios (crear, listar, borrar) y sistema de usuarios, grupos y permisos.

Principales Interfaces de Acceso

1. **Cliente de Línea de Comandos (`hadoop fs`):** El más común y completo. Se ejecuta desde un nodo con Hadoop (ej., NameNode o nodo frontera).
2. **Java API:** Nativo, de alto rendimiento, para aplicaciones Java.
3. **REST API (WebHDFS):** Interfaz HTTP para usar desde otros lenguajes. Rendimiento inferior, no recomendado para operaciones masivas.
4. **NFS Gateway:** Permite montar HDFS como un sistema de archivos local vía NFSv3. Útil para usar utilidades Unix (`ls` , `cat`), pero **solo soporta operaciones de añadir (append)**, no modificaciones.

5. **Librería C (libhdfs)**: Buen rendimiento, pero funcionalidad potencialmente menor que la API Java.

Comandos de Línea de Comandos Principales

- `hadoop fs -mkdir <ruta>` : Crea un directorio.
- `hadoop fs -ls <ruta>` : Lista el contenido de un directorio. `-R` para recursivo.
- `hadoop fs -put <origen_local> <destino_hdfs>` : Copia archivos del sistema local a HDFS.
- `hadoop fs -get <origen_hdfs> <destino_local>` : Copia archivos de HDFS al sistema local.
- `hadoop fs -cat <ruta_archivo>` : Muestra el contenido de un archivo.
- `hadoop fs -cp <origen> <destino>` : Copia archivos dentro de HDFS.
- `hadoop fs -rm <ruta_archivo>` : Elimina un archivo. `-R` para eliminar directorios recursivamente.
- `hadoop fs -mv <origen> <destino>` : Mueve/renombrar archivos dentro de HDFS.
- `hadoop fs -setrep <factor> <ruta>` : Cambia el factor de replicación de un archivo o directorio.

Nota crucial: Es importante distinguir entre el **sistema de archivos local** de la máquina desde donde se ejecuta el terminal (nodo frontera) y el **sistema de archivos distribuido HDFS**. Los comandos `hadoop fs` operan sobre HDFS, no sobre el disco local.

Resumen Detallado: HDFS y YARN en Hadoop

1. Comandos Básicos de HDFS

HDFS (Hadoop Distributed File System) es el sistema de archivos distribuido de Hadoop, **separado del sistema de archivos local** de la máquina.

Comandos Clave y sus Funciones:

- **hadoop fs** : Ejecuta comandos sobre HDFS (no sobre el sistema local).
- **put** : Sube archivos **desde la máquina local a HDFS**. Es necesario copiar primero el archivo a la máquina frontera (o edge node).
- **ls** : Lista el contenido de un **directorio** en HDFS o muestra metadatos de archivos (nombre, permisos, fecha, etc.).
- **cp** : Copia archivos **dentro de HDFS**. No funciona con archivos fuera de HDFS.
- **cat** : Muestra el **contenido de un archivo** (no confundir con **ls**).

Nota práctica: Aunque trabajar por terminal pueda parecer menos intuitivo, es la herramienta principal para administradores de sistemas por su eficiencia y control.

⚙ 2. YARN: El Gestor de Recursos y Procesamiento

2.1 Introducción a YARN

YARN (Yet Another Resource Negotiator) es el "**sistema operativo**" de Hadoop, responsable de gestionar los recursos del clúster y ejecutar aplicaciones de procesamiento de datos.

Problemas que resuelve (presentes en Hadoop v1 con solo MapReduce): 1. **Flexibilidad:** MapReduce restringía el tipo de aplicaciones (ej. procesamiento en tiempo real era difícil). 2. **Eficiencia:** MapReduce es un modelo lento para casos que requieren respuestas rápidas. 3. **Concurrencia:** La ejecución de trabajos era secuencial; un trabajo debía terminar para empezar otro.

Funciones principales de YARN: - Ofrece una **API menos estricta** para aplicaciones (no impone un modelo de procesamiento fijo). - **Gestiona la ejecución** de aplicaciones en el clúster (asigna CPU, memoria). - **Sincroniza ejecuciones simultáneas** (gestión de prioridades y recursos). - **Monitoriza aplicaciones** y garantiza **tolerancia a fallos** (relanza tareas si fallan). - **Gestiona los recursos** disponibles en el clúster.

2.2 Arquitectura de YARN

Concepto Clave: Contenedores

- Un **contenedor** es la **unidad mínima de recursos** (memoria, CPU cores, disco, red) para ejecutar tareas.
- **Ejemplo:** Un contenedor puede tener 4 GB de RAM y 1 core.
- Las **tareas** de las aplicaciones se ejecutan en contenedores. Al terminar una tarea, el contenedor se libera.
- El **tamaño del contenedor** es configurable (por defecto 8 GB). Se ajusta según la potencia de los nodos y la complejidad de las tareas.
- La **capacidad de procesamiento concurrente** del clúster está limitada por el número total de contenedores disponibles.

Componentes de la Arquitectura

Componente	Tipo de Nodo	Función Principal
ResourceManager	Nodo Maestro	Coordina y controla todas las aplicaciones en el clúster. Es el punto único de fallo crítico (sin él, no se ejecutan aplicaciones).
NodeManager	Nodo Worker (en cada nodo)	Monitoriza recursos (CPU/memoria) en su nodo, supervisa contenedores y reporta al ResourceManager. Tolerante a fallos.
ApplicationMaster	Por Aplicación (en un nodo worker)	Negocia recursos con el ResourceManager para una aplicación específica . Se termina cuando la aplicación finaliza.

Subcomponentes del ResourceManager:

1. **ApplicationsMaster:** Gestiona el ciclo de vida de las aplicaciones (distribución, monitorización, liberación de recursos).
2. **Scheduler:** Asigna recursos según políticas configuradas:
 - **Capacity Scheduler** (más común): Crea colas con porcentajes de recursos (ej., cola para servicios críticos, otra para data scientists).
 - **Fair Scheduler:** Asigna recursos equitativamente entre aplicaciones a lo largo del tiempo.
 - **FIFO Scheduler:** Asigna por orden de llegada (el primero en solicitar obtiene los recursos).

2.3 Funcionamiento de YARN: Flujo de Ejecución

1. **Solicitud:** Un cliente contacta al **ResourceManager** para ejecutar una aplicación, enviando el código y los recursos requeridos.
 2. **Asignación:** El **ApplicationsMaster** (dentro del ResourceManager) verifica disponibilidad y prioridad con el **Scheduler**.
 3. **Ejecución:** Se asigna un **ApplicationMaster** específico (en un nodo worker) que gestiona las tareas de esa aplicación, solicitando contenedores a los **NodeManagers**.
 4. **Monitorización:** El ResourceManager y el ApplicationMaster supervisan la ejecución, manejando fallos y liberando recursos al finalizar.
-

Puntos Clave para el Estudio

- **HDFS y el sistema local son distintos:** Usa `put` para subir archivos locales a HDFS.
- **YARN separa el procesamiento del almacenamiento**, permitiendo mayor flexibilidad y concurrencia que MapReduce solo.
- **El ResourceManager es el componente crítico** (maestro) en YARN.
- **Los contenedores son la unidad de ejecución;** su tamaño y cantidad limitan la concurrencia.
- **El flujo de YARN** inicia con una solicitud al ResourceManager, que coordina la ejecución distribuida mediante ApplicationMasters y NodeManagers.

Propósito General: Este texto explica la operación básica de HDFS (comandos) y la arquitectura de YARN, mostrando cómo Hadoop gestiona el almacenamiento distribuido (HDFS) y el procesamiento paralelo y flexible (YARN) para análisis de big data, superando las limitaciones de las primeras versiones basadas solo en MapReduce.

Resumen Detallado: YARN y MapReduce en Hadoop

1. Arquitectura de YARN y Flujo de Ejecución

Proceso de Ejecución en YARN:

1. **Cliente → ResourceManager (RM):** El cliente envía la aplicación solo al RM, sin necesidad de conectarse directamente a los nodos worker.
2. **RM → NodeManager (NM):** El RM solicita a un NM la creación de un contenedor para el ApplicationMaster (AM).
3. **Ejecución del AM:** El AM se ejecuta y gestiona la aplicación: - Solicita recursos (contenedores) al RM. - Controla la ejecución de tareas en los contenedores. - Notifica el estado al RM.
4. **Monitoreo:** Los NM envían información de recursos y estado al RM.

Clave de diseño: Los nodos maestros (RM) realizan pocas operaciones por tarea para permitir escalabilidad, ya que son el cuello de botella potencial del sistema.

2. Introducción a MapReduce

Definición y Características:

MapReduce es un **framework** para procesar grandes volúmenes de datos en paralelo en clústeres de hardware estándar, de manera confiable y tolerante a fallos.

Características principales:

- **Framework:** Maneja complejidades como paralelización, monitoreo y recuperación de errores (90% del esfuerzo en sistemas masivos). El desarrollador solo implementa la lógica de negocio en las funciones `map` y `reduce`.

- **Grandes volúmenes de datos:** Aplica el paradigma "**Divide y Vencerás**":
 - Divide el dataset en fragmentos pequeños.
 - Procesa cada fragmento independientemente.
 - Combina resultados parciales.
- **Procesamiento paralelo y distribuido:** Cada fragmento se ejecuta en diferentes nodos del clúster, gestionado por YARN.
- **Hardware commodity:** Utiliza los mismos servidores del clúster Hadoop.
- **Confiabilidad y tolerancia a fallos:** Recupera tareas fallidas en otros nodos. Es "lento pero seguro", ideal para procesos nocturnos que priorizan la finalización correcta.

Estado actual:

Aunque su uso ha disminuido, es importante entenderlo porque: 1. Su modelo de programación es común en otras herramientas Big Data. 2. Herramientas como **Hive** lo ejecutan internamente (aunque usen interfaces como SQL). 3. Permite depurar problemas en ejecuciones complejas.

Desarrollo: Principalmente en Java, pero también soporta otros lenguajes (C++, Python, scripting) mediante Hadoop Streaming o Hadoop Pipes.

3. Funcionamiento de MapReduce

Las 5 etapas de un trabajo MapReduce:

1. **Envío y distribución:** El cliente envía el trabajo al RM de YARN.
2. **Fase Map:** Procesa fragmentos de datos en paralelo.
3. **Fase Shuffle:** Reorganiza y transmite datos intermedios.
4. **Fase Sort:** Ordena datos por clave.
5. **Fase Reduce:** Agrega y consolida resultados.

Nota: El programador solo implementa las fases **Map** y **Reduce**; las demás son automáticas.

Ejemplo Práctico: Análisis de Cotizaciones Bursátiles

Objetivo: Contar cuántas veces subió la cotización de cada empresa en un dataset histórico (ej: 25 TB, ~276 mil millones de líneas).

Formato de datos:

FechaHora;Empresa;CotizaciónActual;CotizaciónAnterior

Ej: 20/01/2021 11:54:34;SANTANDER;4,54;4,49

Proceso paso a paso:

1. **División (InputFormat):** El fichero se divide en fragmentos manejables.
2. **Fase Map (por fragmento en paralelo):** - Lee cada línea. - Si `CotizaciónActual > CotizaciónAnterior`, emite `[Empresa, 1]`. - **Ejemplo de salida parcial por nodo:**
`SANTANDER, 1 TELEFONICA, 1 APPLE, 1`
3. **Shuffle & Sort (automático):** - Agrupa y ordena por empresa:
`SANTANDER, [1, 1, 1...] TELEFONICA, [1, 1...]`
4. **Fase Reduce:** - Suma los valores por empresa. - Produce el resultado final:
`SANTANDER 3888981 TELEFONICA 3331923`

Representación gráfica: Ilustra cómo los datos se dividen, procesan en paralelo y se consolidan.

4. Consideraciones de Diseño y Limitaciones

Puntos clave:

- **Diseño previo esencial:** MapReduce requiere planificar cómo descomponer el problema en operaciones `map` y `reduce`.
- **Procesamiento paralelo:** Los datos de entrada **se fragmentan y procesan en paralelo**, acelerando el cómputo.
- **Limitaciones:** No todos los problemas son aptos para MapReduce:
 - Problemas que **no permiten división de datos de entrada**.
 - Problemas complejos pueden requerir **varios trabajos MapReduce encadenados** (flujos de ejecución).

- **Filosofía:** Descomponer problemas complejos en subproblemas resolubles con una fase de procesamiento individual (`map`) y una fase de agregación (`reduce`).
-

Autoevaluación (Respuestas):

1. **MapReduce requiere una etapa de diseño previo: Verdadero.**
 2. **Los datos de entrada se separan en fragmentos procesados en paralelo: Verdadero.**
 3. **Hay problemas que no pueden resolverse con MapReduce: Verdadero.**
-

5. Aplicación en el Contexto Empresarial

Casos de uso para Roberto (empresa de transportes):

- Análisis de estilos de conducción.
- Predicción de roturas en vehículos.
- Optimización de rutas en tiempo real con datos de tráfico.

Conclusión: MapReduce, aunque menos usado hoy, es fundamental para entender el procesamiento distribuido en Hadoop y sirve como base para herramientas de alto nivel. Su enfoque en confiabilidad lo hace valioso para procesos batch críticos donde la completitud es prioritaria sobre la velocidad.

Análisis de Cotizaciones con MapReduce - Guía Práctica

Contexto del Problema

Objetivo: Analizar un archivo masivo de datos bursátiles (25 TB, ~276 mil millones de líneas) para contar cuántas veces cada empresa experimentó un incremento en su cotización.

Formato de datos: Cada línea contiene:

```
Fecha_Hora;EMPRESA;Cotización_Actual;Cotización_Anterior
```

Ejemplo: 20/01/2021 11:54:34;SANTANDER;4,54;4,49

Resultado esperado:

```
SANTANDER 3888981
TELEFONICA 3331923
```

Arquitectura MapReduce

Fase MAP (Filtrado y Transformación)

- **Función:** Procesa cada línea individualmente
- **Lógica implementada:** 1. Divide la línea usando `;` como separador 2. Extrae: nombre empresa, cotización actual y anterior 3. **Filtra:** Solo emite resultado si `cotización_actual > cotización_anterior` 4. **Emite:** Pareja `(Nombre_Empresa, 1)`

Ejemplo de procesamiento: - Línea: 20/01/2021 11:54:34;SANTANDER;4,54;4,49 - Como 4,54 > 4,49 → Emite: (SANTANDER, 1) - Línea: 14/05/1995 09:54;TELEFONICA;11,90;12,01 - Como 11,90 < 12,01 → **No emite nada**

Fase REDUCE (Agregación)

- **Función:** Agrupa y suma todos los valores por empresa
- **Mecanismo:** Hadoop agrupa automáticamente todas las parejas con misma clave
- **Proceso:** 1. Recibe: (SANTANDER, [1, 1, 1, 1, ...]) 2. Suma todos los valores del array 3. Emite: (SANTANDER, 3888981)

Implementación en Java (Hadoop)

Clases Principales:

```
// 1. MAPPER
public static class IncreaseQuotationFilterMapper
    extends Mapper<Object, Text, Text, IntWritable> {

    public void map(Object key, Text value, Context context) {
        String[] data = value.toString().split(";");
        float current = Float.parseFloat(data[2]);
        float last = Float.parseFloat(data[3]);
        String company = data[1];

        if (current > last) {
            context.write(new Text(company), new IntWritable(1));
        }
    }
}

// 2. REDUCER
public static class IntSumReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterable<IntWritable> values, Context context) {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        context.write(key, new IntWritable(sum));
    }
}
```

```
}  
}
```

Configuración del Job:

```
public static void main(String[] args) throws Exception {  
    Configuration conf = new Configuration();  
    Job job = Job.getInstance(conf, "QuotationAnalyzer");  
  
    job.setJarByClass(QuotationAnalyzer.class);  
    job.setMapperClass(IncreaseQuotationFilterMapper.class);  
    job.setCombinerClass(IntSumReducer.class); // Optimización local  
    job.setReducerClass(IntSumReducer.class);  
  
    job.setOutputKeyClass(Text.class);  
    job.setOutputValueClass(IntWritable.class);  
  
    FileInputFormat.addInputPath(job, new Path(args[0]));  
    FileOutputFormat.setOutputPath(job, new Path(args[1]));  
  
    System.exit(job.waitForCompletion(true) ? 0 : 1);  
}
```

Ejecución en Cluster Hadoop

Pasos para ejecutar:

1. **Empaquetar:** Crear archivo JAR (quotation-analyzer.jar)
2. **Ejecutar comando:**

```
bin/hadoop jar quotation-analyzer.jar QuotationAnalyzer \  
    /data/markets/quotations/input/allquotations.csv \  
    /data/markets/quotations/output
```

Parámetros del comando:

1. `hadoop jar` → Comando para ejecutar aplicaciones Java en Hadoop
2. `quotation-analyzer.jar` → Archivo JAR con el código
3. `QuotationAnalyzer` → Clase principal (contiene `main()`)
4. **Primera ruta** → Directorio/archivo de entrada
5. **Segunda ruta** → Directorio de salida (no debe existir)

Conceptos Clave de MapReduce

Ventajas Demostradas:

- **Escalabilidad automática:** Hadoop divide el archivo en bloques
- **Procesamiento paralelo:** Múltiples mappers y reducers ejecutándose simultáneamente
- **Tolerancia a fallos:** Hadoop gestiona automáticamente fallos de nodos
- **Abstracción:** El desarrollador solo implementa lógica de negocio (map y reduce)

Flujo de Datos:

```
Archivo de Entrada (276B líneas)
    ↓
  Split en Bloques
    ↓
Mappers Paralelos
    ↓
  Shuffle & Sort
    ↓
Reducers Paralelos
    ↓
  Archivo de Salida
```

Puntos de Atención en la Implementación

Errores Comunes:

1. **NOTA:** Hay un error en el código proporcionado: `SingleCountryData[companyName]` debería ser solo `companyName`
2. **Formato numérico:** Los datos usan coma decimal (4,54) → compatible con `Float.parseFloat()`
3. **Control de excepciones:** En producción, se debe manejar líneas mal formadas

Optimización:

- **Combiner:** Se usa `IntSumReducer` como combiner para reducir tráfico de red
- **Tipos Hadoop:** Se usan `Text` e `IntWritable` en lugar de `String` e `int` para serialización eficiente

Aplicaciones Prácticas

Este patrón (filtrado + conteo) es aplicable a múltiples escenarios: - **Análisis de logs:** Contar errores por tipo - **Procesamiento de clicks:** Contar visitas por página - **Análisis de sensores:** Contar eventos por dispositivo

Conclusión: MapReduce permite procesar datasets masivos escribiendo solo la lógica de transformación (map) y agregación (reduce), mientras Hadoop maneja automáticamente la distribución, paralelización y tolerancia a fallos.